

Practical No. : 7

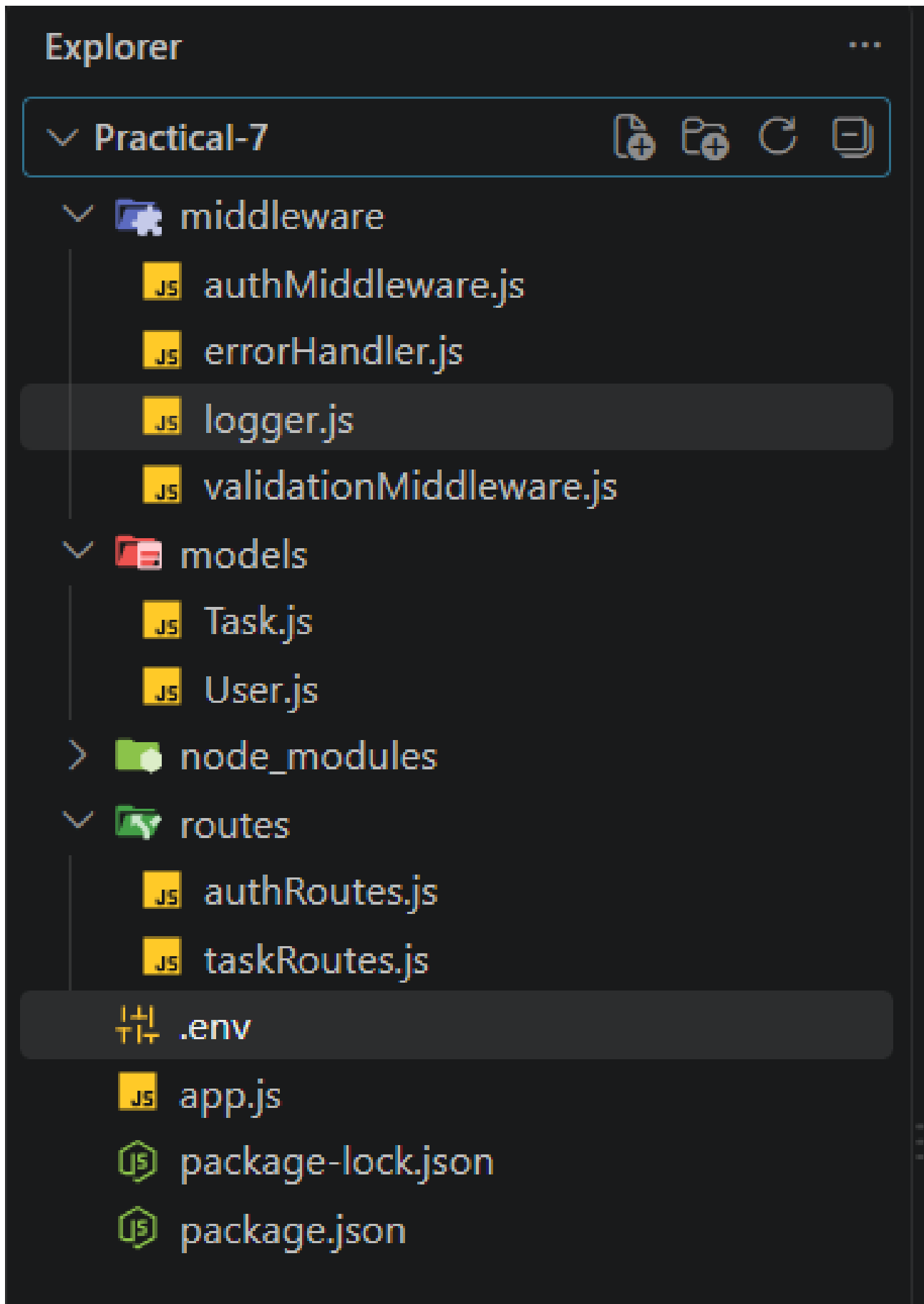
Name : Bhargav Rathod

Enrollment No. : 24CS086

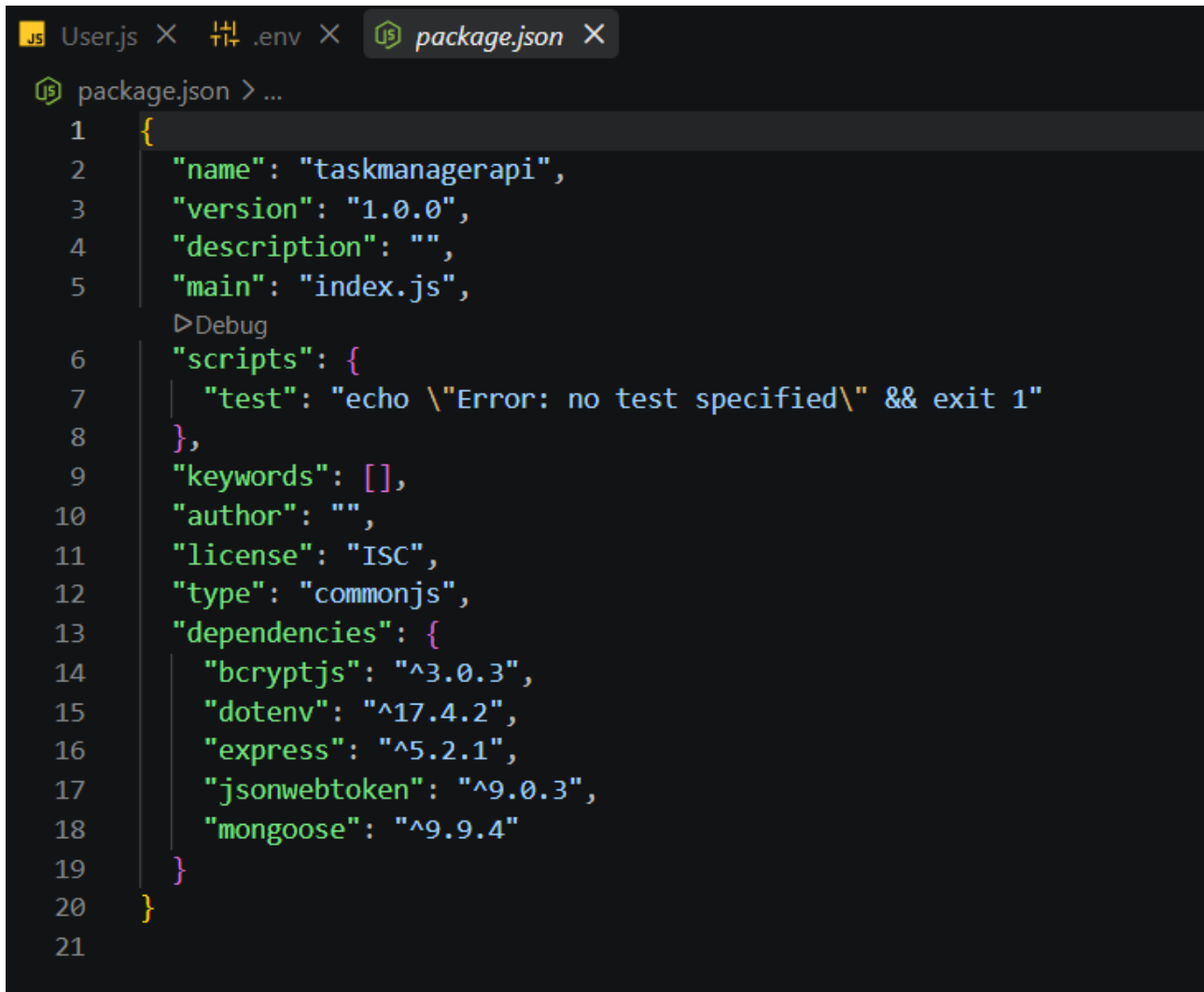
Division : 2

Batch : A2

Screenshot 1: Project Setup / Folder Structure



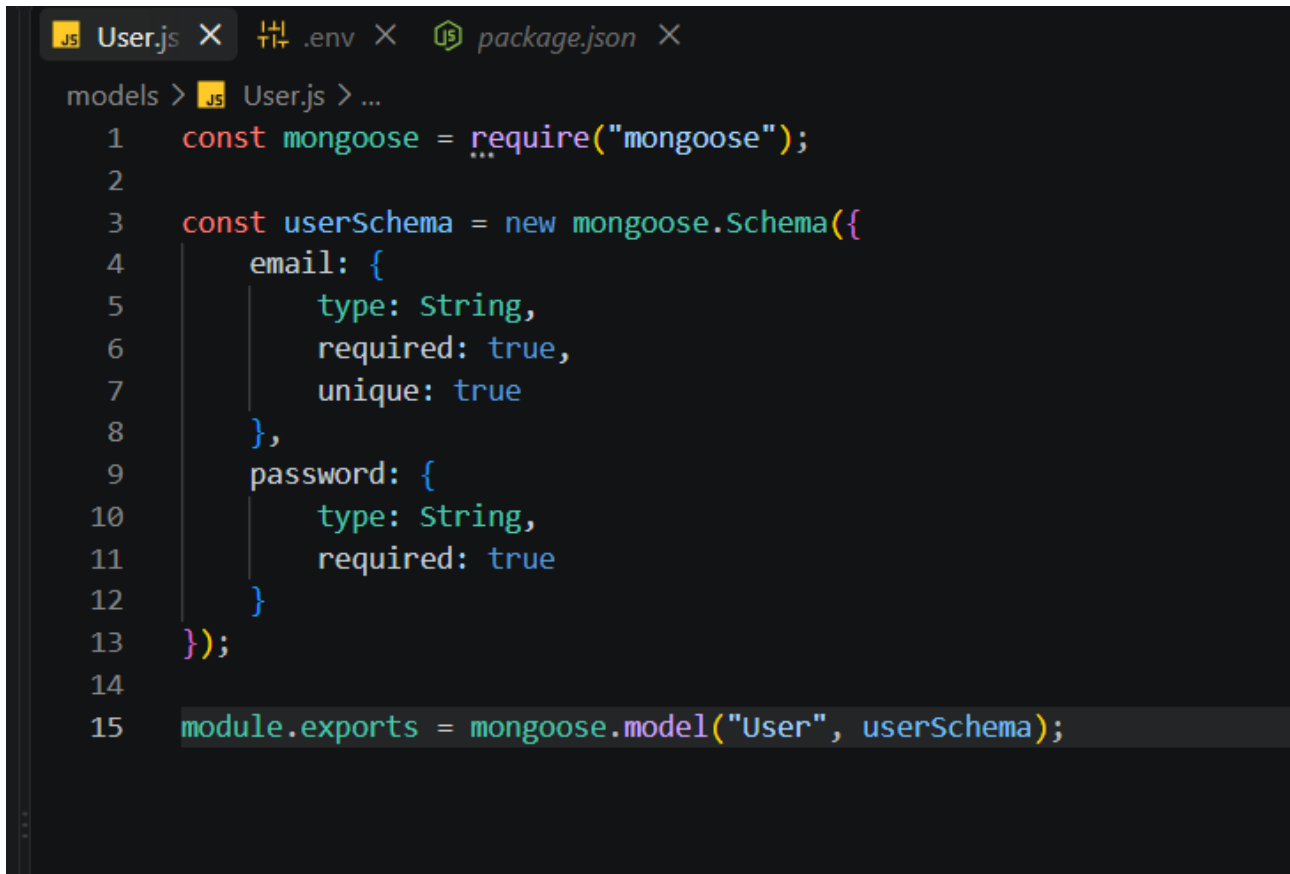
Screenshot 2: Required Packages and Dependencies



The screenshot shows a code editor with three tabs: 'User.js', '.env', and 'package.json'. The 'package.json' tab is active, displaying the following JSON configuration:

```
1  {
2    "name": "taskmanagerapi",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "type": "commonjs",
13   "dependencies": {
14     "bcryptjs": "^3.0.3",
15     "dotenv": "^17.4.2",
16     "express": "^5.2.1",
17     "jsonwebtoken": "^9.0.3",
18     "mongoose": "^9.9.4"
19   }
20 }
```

Screenshot 3: User Schema for Authentication



The screenshot shows a code editor with three tabs: 'User.js', '.env', and 'package.json'. The 'User.js' tab is active, showing the following code:

```
models > JS User.js > ...
1  const mongoose = require("mongoose");
2
3  const userSchema = new mongoose.Schema({
4    email: {
5      type: String,
6      required: true,
7      unique: true
8    },
9    password: {
10     type: String,
11     required: true
12   }
13 });
14
15 module.exports = mongoose.model("User", userSchema);
```

Screenshot 4: User Registration and Password Hashing

```
User.js X authRoutes.js X .env X
routes > authRoutes.js > ...
1  const express = require("express");
2  const bcrypt = require("bcrypt");
3  const jwt = require("jsonwebtoken");
4  const User = require("../models/User");
5
6  const router = express.Router();
7
8
9  // REGISTER USER
10 router.post("/register", async (req, res) => {
11   try {
12     const { email, password } = req.body;
13
14     if (!email || !password) {
15       return res.status(400).json({
16         message: "Email and password are required"
17       });
18     }
19
20     const existingUser = await User.findOne({ email });
21
22     if (existingUser) {
23       return res.status(400).json({
24         message: "User already exists"
25       });
26     }
27
28     const hashedPassword = await bcrypt.hash(password, 10);
29
30     const user = await User.create({
31       email,
32       password: hashedPassword
33     });
34
35     res.status(201).json({
36       message: "User registered successfully",
37       userId: user._id
38     });
39   } catch (error) {
40     res.status(500).json({
41       message: error.message
42     });
43   }
44 }
45 });
46
```

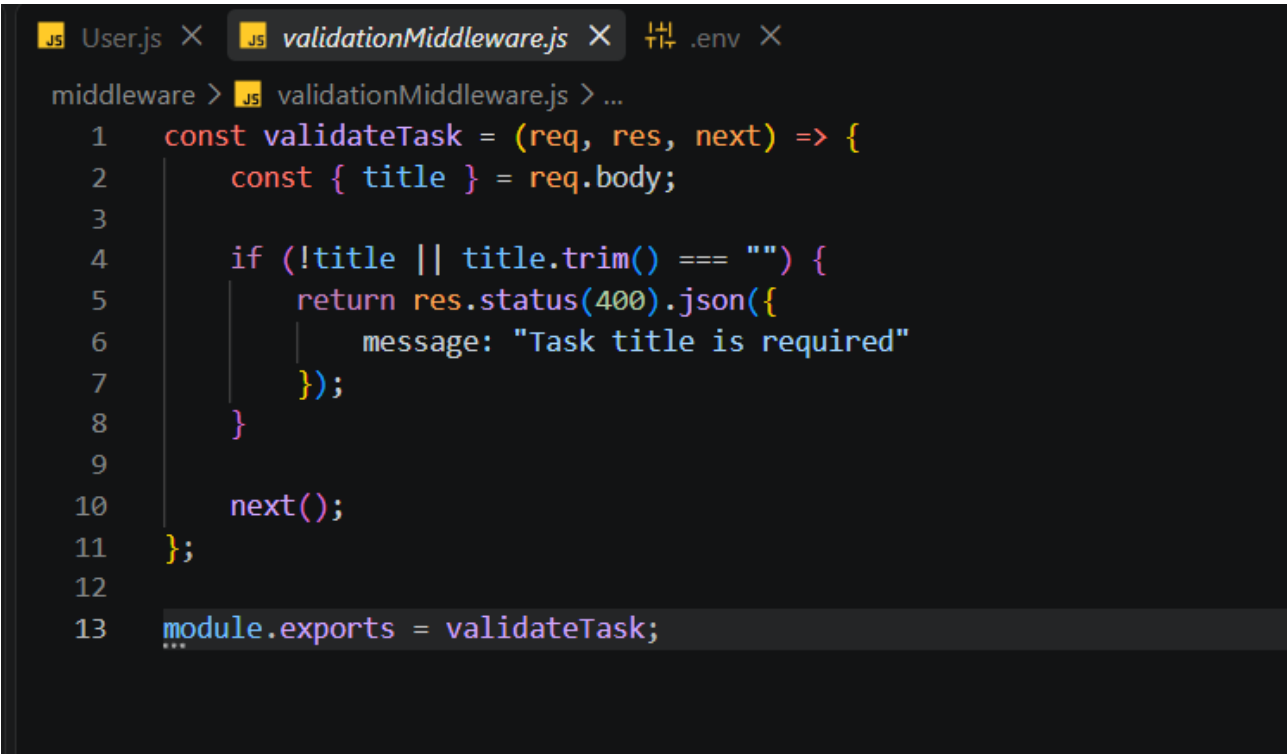
Screenshot 5: User Login and JWT Token Generation

```
User.js X authRoutes.js X .env X
routes > authRoutes.js > ...
47
48 // LOGIN USER
49 router.post("/login", async (req, res) => {
50   try {
51     const { email, password } = req.body;
52
53     if (!email || !password) {
54       return res.status(400).json({
55         message: "Email and password are required"
56       });
57     }
58
59     const user = await User.findOne({ email });
60
61     if (!user) {
62       return res.status(401).json({
63         message: "Invalid email or password"
64       });
65     }
66
67     const isMatch = await bcrypt.compare(
68       password,
69       user.password
70     );
71
72     if (!isMatch) {
73       return res.status(401).json({
74         message: "Invalid email or password"
75       });
76     }
77
78     const token = jwt.sign(
79       { id: user._id },
80       process.env.JWT_SECRET,
81       { expiresIn: "1h" }
82     );
83
84     res.status(200).json({
85       message: "Login successful",
86       token
87     });
88
89   } catch (error) {
90     res.status(500).json({
91       message: error.message
92     });
93   }
94 });
95
96 module.exports = router;
```

Screenshot 6: JWT Authentication Middleware

```
JS User.js × JS authMiddleware.js × .env ×
middleware > JS authMiddleware.js > ...
1  const jwt = require("jsonwebtoken");
2
3  const authMiddleware = (req, res, next) => {
4    try {
5      const token = req.headers.authorization?.split(" ")[1];
6
7      if (!token) {
8        return res.status(401).json({
9          message: "Access denied. No token provided."
10        });
11      }
12
13      const decoded = jwt.verify(token, process.env.JWT_SECRET);
14
15      req.user = decoded;
16
17      next();
18    } catch (error) {
19      return res.status(401).json({
20        message: "Invalid or expired token."
21      });
22    }
23  };
24
25  module.exports = authMiddleware;
```

Screenshot 7: Server-Side Input Validation Middleware



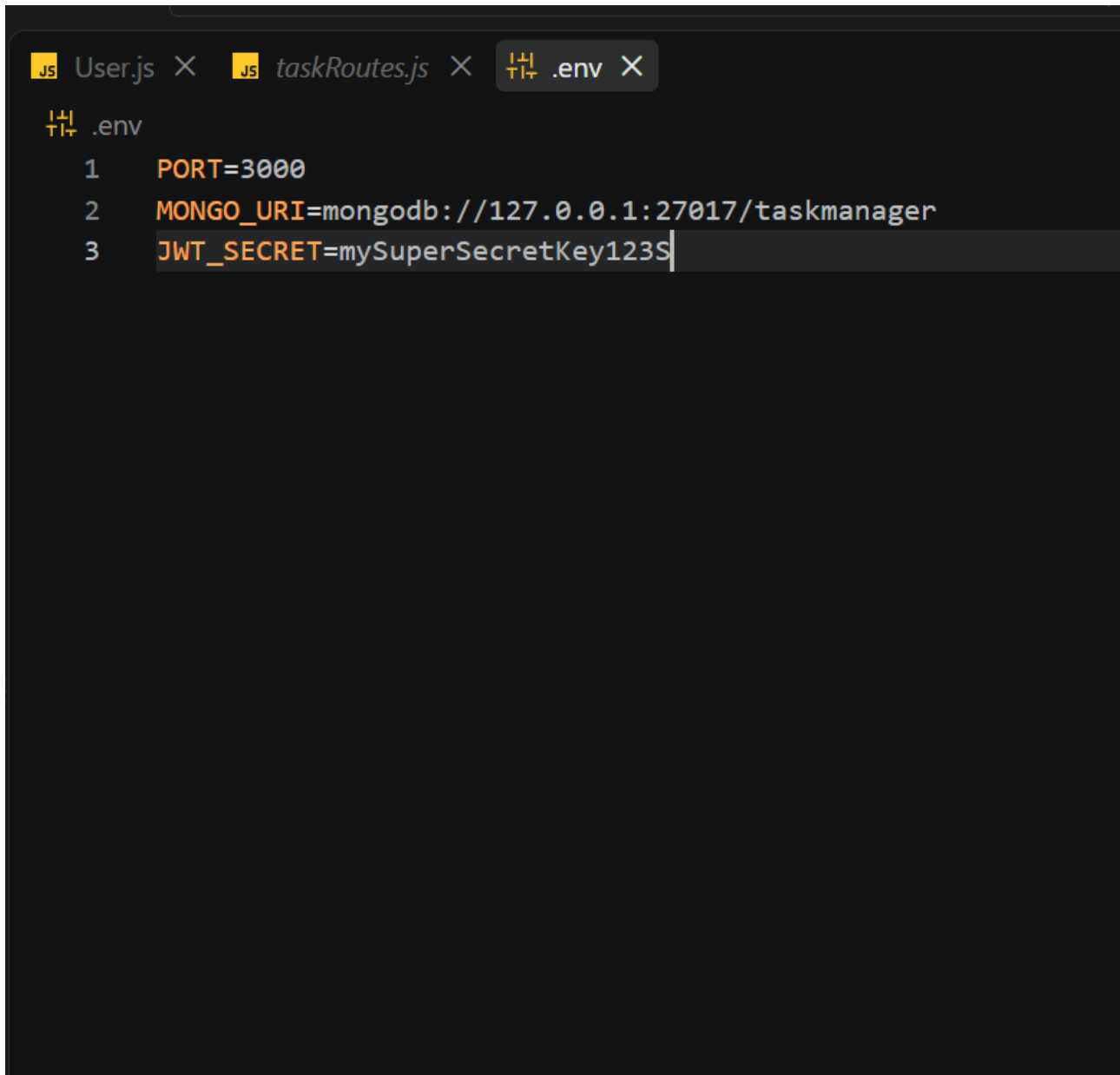
The screenshot shows a code editor with three tabs: 'User.js', 'validationMiddleware.js', and '.env'. The 'validationMiddleware.js' tab is active, displaying a JavaScript function named 'validateTask'. The function takes 'req', 'res', and 'next' as arguments. It extracts the 'title' from 'req.body'. If 'title' is missing or an empty string, it returns a 400 status response with a JSON message: 'Task title is required'. Otherwise, it calls 'next()' to proceed. The function is then exported as 'module.exports'.

```
middleware > validationMiddleware.js > ...
1  const validateTask = (req, res, next) => {
2      const { title } = req.body;
3
4      if (!title || title.trim() === "") {
5          return res.status(400).json({
6              message: "Task title is required"
7          });
8      }
9
10     next();
11 };
12
13 module.exports = validateTask;
```

Screenshot 8: Protected Task Routes Using Authentication Middleware

```
User.js x taskRoutes.js x .env x
routes > taskRoutes.js > ...
1  const express = require("express");
2  const Task = require("../models/Task");
3  const authMiddleware = require("../middleware/authMiddleware");
4  const validateTask = require("../middleware/validationMiddleware");
5
6  const router = express.Router();
7
8
9  // GET ALL TASKS
10 router.get("/", authMiddleware, async (req, res) => {
11   try {
12     const tasks = await Task.find({
13       user: req.user.id
14     });
15
16     res.status(200).json(tasks);
17   } catch (error) {
18     res.status(500).json({
19       message: error.message
20     });
21   }
22 }
23 });
24
25
26 // CREATE TASK
27 router.post(
28   "/",
29   authMiddleware,
30   validateTask,
31   async (req, res) => {
32     try {
33       const { title, description } = req.body;
34
35       const task = await Task.create({
36         title,
37         description,
38         user: req.user.id
39       });
40
41       res.status(201).json({
42         message: "Task created successfully",
43         task
44       });
45     } catch (error) {
46       res.status(500).json({
47         message: error.message
48       });
49     }
50   }
51 );
52
53
54
```

Screenshot 9: Environment Configuration for MongoDB and JWT



The screenshot shows a code editor with three tabs: `User.js`, `taskRoutes.js`, and `.env`. The `.env` tab is active, displaying the following configuration:

```
1  PORT=3000
2  MONGO_URI=mongodb://127.0.0.1:27017/taskmanager
3  JWT_SECRET=mySuperSecretKey123S
```

Screenshot 10: Successful Server and MongoDB Connection

```
PS C:\Users\Bhargav Rathod\OneDrive\Desktop\College\5th Sem\AWDF\Practical\Practical-7> node app.js
◇ injected env (3) from .env // tip: % multiple files { path: ['.env.local', '.env'] }
Server is running on port 3000
MongoDB Connected Successfully
```

Screenshot 11: Successful User Registration

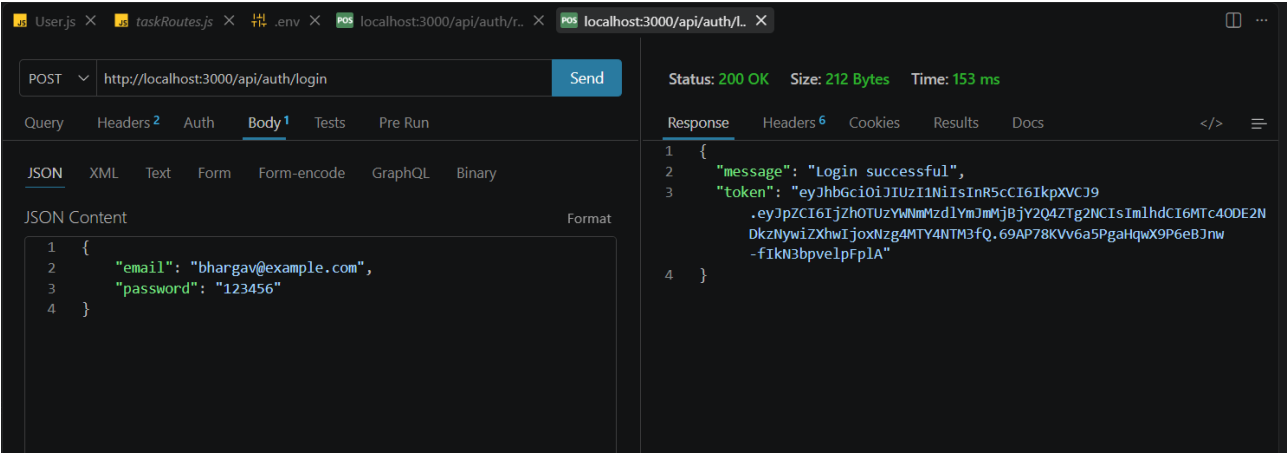
The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: `http://localhost:3000/api/auth/register`
 - Body (JSON):

```
1 {
2   "email": "bhargav@example.com",
3   "password": "123456"
4 }
```
- Response:**
 - Status: 201 Created
 - Size: 78 Bytes
 - Time: 823 ms
 - Body (JSON):

```
1 {
2   "message": "User registered successfully",
3   "userId": "6a953acf37ebbf20ccd8e864"
4 }
```

Screenshot 12: Successful User Login and JWT Token Generation



Screenshot 13: Accessing Protected Task Route Using JWT

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: GET
 - URL: `http://localhost:3000/api/tasks`
 - Tab: **Auth** 1
 - Auth Type: **Bearer**
 - Token: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhOTUzYWVhbnR5cCI6IkpXVCJ9.eyJpZCI6IjZhOTUzYWVhbnR5cCI6IkpXVCJ9`
 - Token Prefix: **Bearer**
- Response:**
 - Status: **200 OK**
 - Size: **255 Bytes**
 - Time: **75 ms**
 - Tab: **Response** 6
 - Content:

```
1  [
2    {
3      "_id": "6a953cdf37ebbf20ccd8e865",
4      "title": "Complete Practical 7",
5      "description": "JWT Authentication and Middleware",
6      "completed": false,
7      "user": "6a953acf37ebbf20ccd8e864",
8      "createdAt": "2026-08-31T08:35:43.540Z",
9      "updatedAt": "2026-08-31T08:35:43.540Z",
10     "_v": 0
11   }
12 ]
```

Screenshot 14: Successful Task Creation

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: http://localhost:3000/api/tasks
 - Body (JSON):

```
1 {
2   "title": "",
3   "description": "Testing validation middleware"
4 }
```
- Response:**
 - Status: 201 Created
 - Size: 300 Bytes
 - Time: 42 ms
 - Body (JSON):

```
1 {
2   "message": "Task created successfully",
3   "task": {
4     "title": "Complete Practical 7",
5     "description": "JWT Authentication and Middleware",
6     "completed": false,
7     "user": "6a953acf37ebbf20ccd8e864",
8     "_id": "6a953cdf37ebbf20ccd8e865",
9     "createdAt": "2026-08-31T08:35:43.540Z",
10    "updatedAt": "2026-08-31T08:35:43.540Z",
11    "__v": 0
12  }
13 }
```

Screenshot 15: Server-Side Validation for Missing Task Title

The screenshot displays a REST client interface with the following details:

- Request:** POST to `http://localhost:3000/api/tasks`. The request body is a JSON object: `{ "title": "", "description": "Testing validation middleware" }`.
- Response:** Status `400 Bad Request`, Size `36 Bytes`, Time `26 ms`. The response body is a JSON object: `{ "message": "Task title is required" }`.

The interface includes tabs for Query, Headers, Auth, Body, Tests, and Pre Run. The Body tab is active, showing the request body. The Response tab is also active, showing the response body. The JSON Content section on the left shows the request body, and the Response section on the right shows the response body.

Screenshot 16: Successful Task Update

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: PUT
 - URL: `http://localhost:3000/api/tasks/6a953cdf37ebbf20ccd8e865`
 - Body (JSON):

```
1 {
2   "title": "Practical 7 Completed",
3   "description": "JWT Authentication, MongoDB and Middleware"
4   ,
5   "completed": true
6 }
```
- Response:**
 - Status: 200 OK
 - Size: 309 Bytes
 - Time: 82 ms
 - Body (JSON):

```
1 {
2   "message": "Task updated successfully",
3   "task": {
4     "_id": "6a953cdf37ebbf20ccd8e865",
5     "title": "Practical 7 Completed",
6     "description": "JWT Authentication, MongoDB and Middleware",
7     "completed": true,
8     "user": "6a953acf37ebbf20ccd8e864",
9     "createdAt": "2026-08-31T08:35:43.540Z",
10    "updatedAt": "2026-08-31T08:45:02.545Z",
11    "__v": 0
12  }
13 }
```

Screenshot 17: Final Verification After Task Deletion

